

OrchANN: Hierarchical Orchestration for Skewed Out-of-Core Vector Search

Lizheng Chen^{1†}, Pinhuan Wang^{1†}, Shaonan Ma², Jie Zhang³, Qing Wang¹, Zhengyi Yang⁴,
Heng Zhang⁵, Mingxing Zhang⁶, Fan Yang⁷, Guihai Chen¹, Chen Tian¹, Chengying Huan¹

¹Nanjing University ²9#AISoft (QiyuanLab) ³Peking University ⁴University of New South Wales
⁵Institute of Software, Chinese Academy of Sciences ⁶Tsinghua University ⁷Huawei

Abstract

At billion scale, approximate nearest neighbor search (ANNS) often becomes an out-of-core problem: the full vector collection and index structures exceed memory capacity, making query performance dominated by SSD accesses and DRAM-SSD data movement. Existing systems struggle to strike a balance between accuracy and efficiency: physical-overlap methods replicate vectors or index entries across partitions, enlarging the SSD-resident index and incurring extra I/O; quantization-based methods reduce memory usage, but their approximate distances are less reliable and often require costly raw-vector reranking from SSD to preserve recall.

We present OrchANN (**Or**chestrated **AN**N Search), an out-of-core ANNS engine that orchestrates query routing, partition access, and query execution under tight memory constraints. OrchANN stores each cluster as a disjoint SSD partition with scale-aware indexes, while a memory-resident graph abstraction provides logical overlap before SSD access. During serving, OrchANN uses query hotness and cluster priorities from the graph abstraction to prune low-value clusters and improve access locality. Across five datasets under strict memory constraints, OrchANN delivers up to 17.2× higher QPS and 25.0× lower latency than state-of-the-art baselines, while preserving accuracy.

1 Introduction

Approximate nearest neighbor search (ANNS) is a core primitive in modern AI and data-intensive systems, including vector databases [39, 46], recommendation engines [9, 35, 38], web search [59], and large language model (LLM) inference [30, 58]. In particular, LLM services such as ChatGPT [40] and Claude [1] rely on vector search for retrieval-augmented generation (RAG) [10, 28, 42], context reuse [2, 55], agentic or knowledge-intensive retrieval [18], and sparse-attention serving [8, 11]. In all these settings, ANNS retrieves a small set of vectors relevant to a query embedding, enabling downstream systems to operate on query-relevant data without exhaustive scans over massive vector collections.

As vector collections grow to billion scale, raw vectors and high-quality indexes can exceed available DRAM capacity, making ANNS an out-of-core problem. Out-of-core ANNS therefore places vectors and most index structures on SSDs,

while keeping only compact routing or filtering metadata in memory [4, 6, 17, 20, 27, 48]. This setting is especially relevant to LLM services, where vector search has become a critical component for retrieval and context management; production-style systems such as OpenAI Vector Store [37] expose vector storage as a first-class service for large document collections. At the same time, GPU HBM in these services is primarily consumed by model-side resources such as LLM weights, KV cache, and activation buffers. Some recent systems explore CPU–GPU–SSD collaboration for billion-scale ANNS [22, 44]. However, they assume that GPU HBM can be provisioned for vector search, which may be difficult in RAG and agentic-AI serving [24, 28, 51, 56]. This paper therefore targets the widely deployable CPU–SSD out-of-core setting, where DRAM is reserved for compact guidance and runtime metadata, while large vectors and local indexes remain SSD-resident.

Existing CPU–SSD out-of-core ANNS systems are mainly organized as graph-based or partition-based indexes. Graph-based systems such as DiskANN [20], Starling [48], and PipeANN [17] store most graph records on SSD and reduce random I/O through memory-resident metadata, graph reordering, or pipelined traversal. Partition-based systems such as SPANN [4] divide vectors into Inverted File (IVF) posting lists or clusters [21], search only a subset of promising partitions, and often rely on an in-memory routing layer to select them. Both approaches make billion-scale search feasible under limited DRAM, but high-recall searches still require boundary reachability: the ability to reach neighbors separated by graph, shard, or partition boundaries.

Existing designs commonly support boundary reachability through either physical overlap in the SSD index or quantized search with raw-vector reranking. For graph-based indexes, physical overlap is realized through graph connectivity: DiskANN builds local graphs from overlapping build partitions and merges their edges into a global SSD-resident graph; at query time, reachability is provided by SSD-resident edges rather than vector replicas. For partition-based indexes, physical overlap is realized through replication: SPANN places boundary vectors in multiple posting lists so that a query can reach them with only a few probes. These mechanisms improve reachability but often increase out-of-core overhead through random graph page accesses, replicated entries, or larger posting lists. Another approach

[†]Equal contribution.

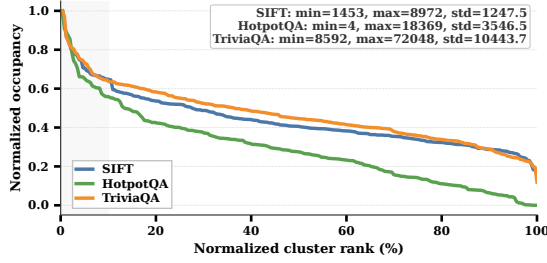


Figure 1. Skewness on SIFT, HotpotQA, and TriviaQA.

is quantized search with raw-vector reranking: systems use Product Quantization (PQ) [21] or Inverted File with Product Quantization (IVFPQ) [21] to maintain compact representations in memory, score candidates approximately, and then fetch full vectors from SSD for exact reranking [20, 44]. This reduces resident memory, but high-recall search still depends on costly raw-vector reranking, turning quantization error into additional SSD traffic.

Design Principle: locality-preserving reachability. The limitation above exposes a framework question: high recall needs boundary reachability, yet placing this reachability in the SSD layout or relying on raw-vector reranking introduces extra SSD work. The system must preserve reachability to neighbors across partition or storage boundaries without sacrificing local and bounded SSD accesses.

Our measurements suggest that disjoint SSD partitions with partition-local indexes provide a better out-of-core substrate. This substrate preserves physical locality: each vector remains in one SSD partition, and SSD search touches only selected partition-local data. As described in Section 3.1, one instantiation with graph-style local indexes preserves coarse I/O locality while avoiding exhaustive scans.

Our design. The architectural insight behind our design is to *decouple physical locality from logical connectivity*. A compact memory-resident graph abstraction provides logical connectivity by routing queries to relevant partitions before SSD access. The SSD layer preserves physical locality by searching only selected non-replicated partitions.

Challenge I: scale-aware partition search. Once boundary recovery is moved to memory, the remaining question is how to search selected SSD partitions efficiently under skewed cluster sizes. As shown in Figure 1, IVF partitioning produces long-tailed clusters even on SIFT (1,453–8,972 vectors, std. 1,247.5) and becomes much more skewed on TriviaQA (8,592–72,048 vectors, std. 10,443.7). Such skewness makes a fixed local index inefficient: tiny partitions may be cheaper to scan than to index, while large partitions can impose high metadata and memory pressure, as quantified in Section 4.3. This calls for stable SSD partitions with local access paths adapted to partition scale and memory pressure.

Challenge II: utility-aware out-of-core execution. Beyond choosing how to search each selected partition, an out-of-core engine must also decide which partition accesses are worth issuing. The memory-resident graph abstraction

exposes partition-level signals before SSD access, and serving workloads repeatedly visit a small subset of partitions, as shown in Figure 6a. However, frequently visited partitions are not necessarily useful for every query, and blindly searching them can still waste SSD I/O. Coarse partition-level signals must therefore be translated into fewer unnecessary out-of-core accesses without sacrificing recall. The challenge is to distinguish useful partition accesses from redundant ones before local search and to organize the remaining accesses for better page reuse.

We present OrchANN (**Or**chestrated **AN**N Search), a hierarchical out-of-core ANNS engine that coordinates routing, indexing, and execution across memory and SSD. To preserve locality while maintaining reachability, OrchANN stores vectors in disjoint SSD partitions and maintains a graph abstraction composed of centroid anchors and representative vectors. To handle skewed partitions, the SSD layer searches partitions with local indexes adapted to partition scale and memory pressure. To reduce redundant out-of-core work, OrchANN uses per-query partition signals to refresh the graph abstraction, prioritize useful partitions, prune low-value accesses, and schedule queries with similar partition visits close together. Together, these mechanisms improve routing precision, reduce redundant SSD searches, and increase temporal locality.

This paper makes the following contributions:

- **A decoupled out-of-core ANNS architecture.** We identify that existing high-recall mechanisms incur extra SSD cost: physical overlap enlarges the SSD-resident index or increases random graph accesses, while quantized search requires full-vector SSD reads for reranking. We propose to decouple physical locality from logical connectivity: raw vectors are kept in non-replicated SSD partitions, while boundary reachability is provided through memory-resident logical overlap.
- **Shape- and scale-aware partition search.** We design OrchANN’s hierarchical search structure, where a compact graph abstraction with representative vectors exposes partition-level evidence before SSD access. The SSD layer then searches selected partitions using local indexes adapted to partition scale and memory pressure.
- **Access-hotness-aware execution.** We design runtime optimizations that turn partition-level access hotness into out-of-core gains. OrchANN refreshes the graph abstraction toward frequently useful regions, prioritizes and prunes candidate partitions using per-query graph evidence, and schedules queries with similar partition visits to improve reuse among remaining SSD accesses.
- **Implementation and evaluation.** We implement OrchANN as a CPU–SSD ANNS engine and evaluate it against representative graph-based, partition-based baselines. Under memory constraints, OrchANN achieves up to 17.2× higher QPS and 25.0× lower latency while preserving recall.

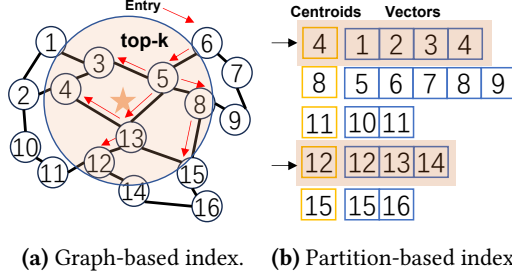


Figure 2. Two basic out-of-core ANNS organizations. A graph index searches by traversing a proximity graph from an entry point toward the query region, while a partition index first selects promising partitions and then searches only the corresponding posting lists or clusters.

2 Background

2.1 Out-of-Core ANNS Architectures

ANNS systems reduce the search space mainly through *graph-based* or *partition-based* indexing. In a graph-based index [6, 17, 20, 27, 48], vectors are organized as a proximity graph, and search starts from one or more entry points and greedily traverses graph edges toward the query-relevant region, as illustrated in Figure 2a. DiskANN [20] is a representative out-of-core graph system: it keeps PQ-compressed vector representations in DRAM for distance estimation, while storing the graph and full-precision vectors on SSD. This design preserves high recall, but SSD-resident graph traversal may still introduce many random page accesses.

A partition-based index [4, 21, 44, 61] instead divides the dataset into clusters or posting lists, typically using IVF or k -means, and searches only a subset of partitions per query. As shown in Figure 2b, the system scans compact partition summaries, ranks partitions by their distance to the query, and searches the nearest vector lists. Systems such as SPANN [4] and RUMMY [61] follow this direction. This design naturally bounds the out-of-core search footprint, but partitioning also weakens connectivity across partition boundaries: true neighbors of a query may be split across multiple partitions.

Some graph-based out-of-core systems also use a small memory-resident graph routing layer above SSD-resident data [17, 48]. Its role is mainly to obtain query-near entry points, rather than to explicitly provide logical overlap among disjoint physical partitions. Existing routing nodes are often centroids [44] or sampled vectors [48]. However, centroids are weak at describing off-center regions in high-dimensional partitions, while sampled vectors may provide closer graph entries but are noisy for partition-level decisions and can mislead the system toward less useful SSD regions (Section 6.2).

2.2 Physical Overlap and Quantized Search

A central challenge in out-of-core ANNS is how to preserve accuracy when true neighbors are separated by graph, shard,

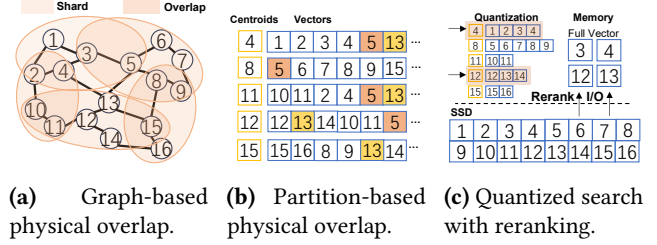


Figure 3. Existing mechanisms for supporting high recall in out-of-core ANNS. Figures 3a and 3b encode boundary reachability in the SSD-resident index through graph connectivity or boundary-vector replication, while Figure 3c stores quantized representations in memory and fetches full vectors from SSD for reranking.

or partition boundaries. Existing systems usually address this problem through the following two mechanisms.

The first direction is physical overlap. In graph-based systems, physical overlap may appear during construction. For example, DiskANN [20] builds a large SSD-resident graph from overlapping partitions: a vector can participate in multiple build partitions, local graphs are constructed under memory limits, and their edges are then merged into a global graph. At search time, cross-boundary reachability is preserved by SSD-resident graph edges, as illustrated in Figure 3a. This helps recover neighbors across storage regions, but may also lead to random accesses across distant SSD pages. In partition-based systems, overlap is more explicit in the index layout. As shown in Figure 3b, SPANN [4] replicates boundary vectors into multiple posting lists, so that a query can find neighbors across partition boundaries even when it probes only a few lists. Such replication improves boundary robustness, but it also enlarges posting lists and increases the amount of SSD-resident data that must be stored.

The second direction is quantized search with reranking. As shown in Figure 3c, compact PQ codes are kept in memory for approximate scoring, while selected full vectors are fetched from SSD for exact reranking. This approach reduces resident memory, but quantized distances are lossy. Therefore, high-recall search often still requires raw-vector reranking, which introduces additional SSD reads.

2.3 Limitations of Prior Works

The mechanisms in Figure 3 support high recall, but they add work to the out-of-core layer. Physical overlap places boundary reachability in the SSD-resident index, which leads to replicated entries, larger posting lists, or random graph accesses. Quantized search keeps the resident index compact, but approximate quantized scores still require many raw-vector reads from SSD for exact reranking at high recall.

We quantify this tradeoff on DEEP100M under Recall@10 metric. The two mechanisms show different failure modes. For SPANN, higher recall quickly turns into much heavier

SSD traffic: increasing Recall@10 from 0.928 to 0.982 raises page accesses from 584 to 1,905 per query, more than 3.2 $\times$ higher, because replicated posting lists enlarge data that may be searched. For IVFPQ, quantized search has the opposite limitation: increasing nprobe from 8 to 128 brings a small Recall@10 improvement, from 0.411 to 0.436, while reducing QPS by 10.4 $\times$. To reach high recall, IVFPQ must fetch raw vectors for reranking; reranking 1,000 vectors/query reaches 0.982–0.991 Recall@10, but introduces hundreds to more than one thousand page-level reads for reranking per query.

These observations point to a different design direction: preserving high recall without enlarging SSD partitions or issuing large amounts of full-vector reads for reranking.

3 Motivation and Design Rationale

The discussion in Section 2.3 shows that existing designs maintain high recall in two costly ways: physical overlap encodes boundary reachability in the SSD-resident index, while quantized search relies on raw-vector reranking. This raises two questions: how can boundary reachability be represented efficiently, and how should partition-local search be organized under skewed workloads? Once the SSD layer is partitioned, we refer to boundary reachability as cross-partition connectivity: reaching relevant partitions even when true neighbors are split across partition boundaries.

3.1 Placement of Cross-Partition Connectivity

We begin with the placement of cross-partition connectivity. In an out-of-core setting, encoding such connectivity directly in the SSD-resident layout can weaken partition locality, while relying only on partition-local search makes boundary-crossing neighbors difficult to recover.

To understand this tradeoff, we examine three architectures under out-of-core memory limits: partition-only IVF search, a monolithic graph-based layout [31], and a disjoint-partition layout with local graph indices inside each partition. The detailed I/O breakdown is reported later in Section 6.2. On SIFT, the disjoint-partition local-index layout reduces page misses to 38.4/query under a 90% memory limit, compared with 1009.4 for IVF and 498.4 for the monolithic graph. Under a tighter 50% limit, it still lowers misses to 132.0/query, compared with 1865.6 and 1392.1, respectively. These results suggest that partition locality should be combined with partition-local indexing, but not through a monolithic disk-resident graph or physical replication across partitions.

Instead, the physical layout should remain partitioned and non-replicated, while cross-partition connectivity is represented logically in memory. This separates two roles: the memory-resident layer identifies promising partitions, and the SSD-resident layer performs local search within them.

Implication 1: logical overlap over non-replicated partitions. High recall should not require duplication or full-vector reranking. Boundary reachability should be provided

through memory-resident logical overlap over non-replicated partitions, with local search adapted inside each partition.

3.2 Shape- and Scale-Aware Search

Once cross-partition connectivity is represented in memory, the next question is how this separation remains effective under skewed data. The memory-resident layer must provide reliable partition-level guidance before SSD access, while the SSD-resident layer must search partitions with very different sizes and access costs without repartitioning.

Shape-aware guidance. A compact memory-resident routing layer cannot rely only on centroids, as in prior designs [44]. Although centroids are small and stable, they provide only a coarse summary of each partition. In our measurement on the largest TriviaQA cluster, most vectors are not close to the centroid: their distances to the centroid are mainly concentrated between 0.43 and 0.48, with the peak around 0.45, and only a small fraction of vectors appear below 0.40. This indicates that dense semantic partitions can have large regions far from the centroid, where relevant query results may still appear. Random samples [48] can cover more such regions, but they are query agnostic and may spend routing capacity on areas that rarely help future queries.

Implication 2: shape-aware partition guidance. The memory-resident routing layer should keep centroid anchors while adding representative vectors that capture off-center partition shape before SSD access.

Scale-aware local search. After the memory-resident layer identifies candidate partitions, the system must decide how to search each partition efficiently. Figure 1 shows that IVF partitioning produces long-tailed cluster sizes across feature and semantic workloads. Even on SIFT [21], cluster sizes range from 1,453 to 8,972 vectors (std. = 1,247.5). The skewness becomes more pronounced on semantic QA workloads: with BGE embeddings [52], HotpotQA [54] contains clusters ranging from a few vectors to more than 18K vectors, while TriviaQA [26] contains clusters ranging from thousands to more than 70K vectors. This long-tailed distribution means that out-of-core ANNS must serve partitions with different search costs, metadata footprints, and page-cache behavior.

A natural alternative is to rebalance partitions, but this is not practical in the out-of-core regime. Rebalancing methods such as Quake [33] and RUMMY [61] are effective for reducing imbalance, but they assume that data movement and index maintenance are affordable. In an SSD-resident setting, repartitioning requires moving vectors, rewriting local index files, and rebuilding affected structures, causing large write amplification and disrupting serving. For example, rebalancing only 1% of a billion-scale dataset already requires rewriting millions of vectors and can take more than five hours in our storage setup.

Implication 3: stable partitions with adaptive local search. Partition boundaries should remain stable on SSD,

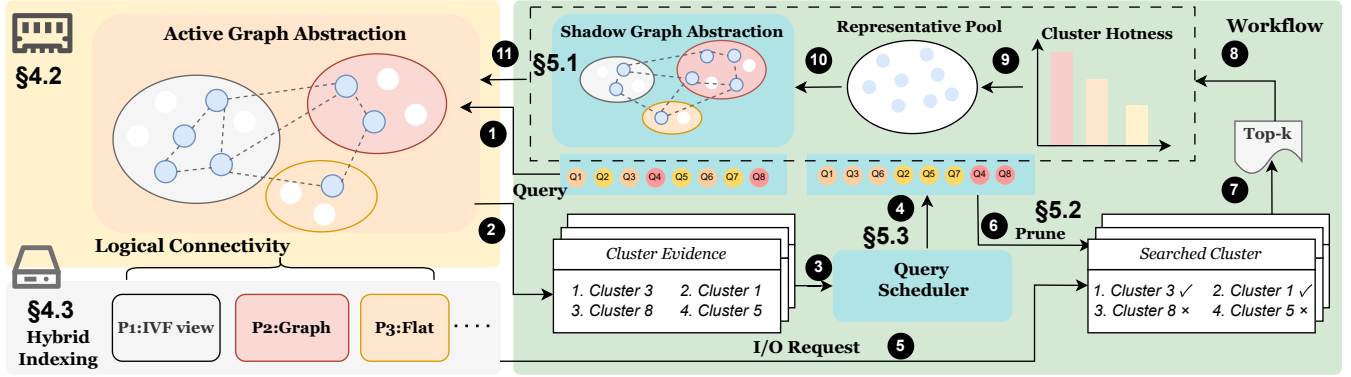


Figure 4. Overview of OrchANN, showing its hierarchical structure and query-processing workflow.

while the local search structure for each partition adapts to its scale and memory pressure.

3.3 Design Implications

The observations above lead to the hierarchical design of OrchANN. First, OrchANN preserves physical locality by keeping SSD partitions non-replicated, and provides logical connectivity through the memory-resident graph abstraction. Second, its routing layer combines centroid anchors with representative vectors to capture partition shape. Third, its SSD-resident layer keeps partition boundaries fixed but adapts local access paths to partition scale. These requirements motivate the architecture in Section 4, while the cluster evidence exposed by this architecture further enables the execution-layer optimizations in Section 5.

4 OrchANN: Hierarchical Out-of-Core ANNS

4.1 OrchANN Overview

To improve the accuracy-memory tradeoff, we introduce OrchANN, a hierarchical out-of-core ANNS engine that decouples physical locality from logical connectivity. Figure 4 illustrates the overall workflow. OrchANN first partitions the vector set into disjoint clusters with k -means, assigning each vector to exactly one physical partition. It then builds a compact memory-resident graph abstraction $\mathcal{GA}$ using cluster centroids and selected data proxies, which characterize each cluster from multiple directions and radii (Section 4.2). Finally, OrchANN constructs an independent local search structure for each out-of-core cluster. This design keeps raw vectors and local indices non-replicated on SSD, while representing cross-partition connectivity logically in memory (Section 4.3).

Workflow. Given a query batch, OrchANN first traverses the active memory-resident graph abstraction $\mathcal{GA}$ for each query and maps the reached probe nodes to clusters, producing a cluster-evidence list before SSD access. The execution

layer then schedules queries with similar predicted cluster visits, prunes low-value candidate clusters, and issues I/O requests only to the remaining selected partitions. For each selected partition, OrchANN invokes its scale-aware local search structure, such as Flat, graph, or auxiliary IVF, and merges local results into the global top- k queue. During serving, lightweight cluster-access statistics are recorded to update cluster hotness, refresh representative vectors on a shadow graph abstraction, and periodically publish the refreshed abstraction for future queries.

4.2 The Memory-Resident Graph Abstraction

OrchANN maintains a compact memory-resident graph abstraction $\mathcal{GA}$ to provide logical connectivity across disjoint SSD partitions before local search. $\mathcal{GA}$ employs a navigable search graph over partition representatives: the graph enables efficient in-memory traversal, while the representatives determine which partition evidence is exposed before SSD access. OrchANN, therefore, represents each cluster with a stable centroid anchor and selected representative vectors, which are connected by proximity edges and traversed entirely in memory before out-of-core local search. Given the disjoint cluster set $\mathcal{C} = \{C_i\}_{i=1}^n$ with centroids $\{c_i\}_{i=1}^n$, the node set of $\mathcal{GA}$ consists of all centroids and selected representative vectors:

$$\mathcal{V}_{\mathcal{GA}} = \{c_i\}_{i=1}^n \cup \bigcup_{i=1}^n R_i, \quad R_i \subseteq C_i.$$

Here, each c_i provides stable global coverage, while R_i contains representative vectors selected from cluster C_i by Algorithm 1 to capture off-center partition structure from diverse directions and radii. During initialization, OrchANN inserts all centroids into $\mathcal{GA}$ and selects the initial representative set R_i for each cluster using Algorithm 1. Each node stores the vector representation needed for in-memory distance computation, together with a lightweight reference to its cluster ID and local vector ID. Thus, $\mathcal{GA}$ can be traversed

Algorithm 1: Representative Vector Selection for Graph Abstraction

Input: cluster C_i , centroid c_i , representative-vector budget b , candidate cap M , radius quantile ρ , score weights α, β

Output: representative vector set R_i

```

1  $P \leftarrow \text{DETERMINISTICSAMPLE}(C_i, M);$ 
2 foreach  $x \in P$  do
3    $r_x \leftarrow \|x - c_i\|_2;$ 
4    $u_x \leftarrow (x - c_i) / \max(r_x, \epsilon);$ 
5 end
6  $Q \leftarrow \{x \in P \mid r_x \geq \text{QUANTILE}(\{r_y : y \in P\}, \rho)\};$ 
7  $R_i \leftarrow \{\arg \max_{x \in Q} r_x\};$ 
8 while  $|R_i| < b$  and  $Q \setminus R_i \neq \emptyset$  do
9   foreach  $x \in Q \setminus R_i$  do
10     $D(x, R_i) \leftarrow 1 - \max_{y \in R_i} \langle u_x, u_y \rangle;$ 
11     $\text{score}(x) \leftarrow \alpha \cdot \text{NORM}(r_x) + \beta \cdot D(x, R_i);$ 
12  end
13   $R_i \leftarrow R_i \cup \{\arg \max_{x \in Q \setminus R_i} \text{score}(x)\};$ 
14 end
15 return  $R_i;$ 

```

entirely in memory, while raw vectors and local indices remain non-replicated on SSD.

Algorithm 1 selects representative vectors that complement the centroid of each cluster. It first samples a bounded candidate pool and computes each candidate's radius and normalized direction relative to the centroid. Since the centroid already represents the central tendency of the cluster, the algorithm uses the radius quantile ρ to focus on off-center candidates that better capture cluster shape. It initializes R_i with the farthest candidate and then greedily selects up to b vectors using a score that combines two signals. The radius term favors candidates that extend coverage beyond the centroid, while the directional-diversity term prevents the selected vectors from collapsing into the same direction. The resulting set gives $\mathcal{GA}$ a compact, shape-aware summary of each physical partition.

4.3 Out-of-Core Local Search Structures

After the graph abstraction identifies the out-of-core partitions, OrchANN has the flexibility to choose the local search structure for each partition independently. As shown in Figure 5, the best structure changes with cluster scale under the same accuracy target: Flat scan is preferable for tiny clusters because it avoids index metadata and random traversal; graph search serves as the primary high-accuracy structure for most clusters; and a compact intra-cluster IVF view becomes useful for very large clusters when graph metadata creates page-cache pressure. Therefore, OrchANN assigns each cluster a scale-aware primary local structure and, for

selected clusters, optionally builds a memory-efficient IVF view.

Scale-aware primary local structure. OrchANN first assigns a primary local structure to every cluster. If the cluster size is below a threshold θ_{flat} , OrchANN uses a Flat structure that only stores the vector range and ID mapping. Query processing then performs a short sequential scan inside the cluster, avoiding unnecessary graph metadata for the long tail of tiny partitions. For all remaining clusters, OrchANN uses a graph index by default. Graph search avoids exhaustive scans while preserving high recall, and the active graph working set remains small enough to benefit from page-cache reuse.

Budget-aware auxiliary IVF view. For very large clusters, the graph index can become inefficient under a tight memory budget: even if graph search reduces distance computations, its metadata and neighbor traversal may touch many random pages and evict useful structures from the page cache. To handle this case, OrchANN builds an additional memory-efficient auxiliary IVF view for the largest clusters.

Specifically, OrchANN sorts clusters by size as $|C_{(1)}| \geq |C_{(2)}| \geq \dots \geq |C_{(n)}|$ and reserves a fraction $\rho_{\text{auxiliary}}$ of the serving memory budget B_{serv} for auxiliary-IVF directories. It then selects the largest prefix that fits this budget:

$$m = \max \left\{ t : \sum_{j=1}^t \hat{M}_{\text{auxiliary}}(C_{(j)}) \leq \rho_{\text{auxiliary}} B_{\text{serv}} \right\}.$$

OrchANN then builds auxiliary IVF views for $C_{(1)}, \dots, C_{(m)}$. Here, $\hat{M}_{\text{auxiliary}}(C_i)$ estimates the resident memory cost of the auxiliary IVF view for cluster C_i . This rule uses one tunable parameter, $\rho_{\text{auxiliary}}$, while the actual number of auxiliary-indexed clusters is determined by the memory budget and the cluster-size distribution.

The auxiliary IVF view does not replicate raw vectors. Instead, it stores a compact intra-cluster IVF directory whose posting lists reference the same raw-vector range. During query execution, the graph index remains the default local structure. Under high memory pressure, or when a cluster is reached as a lower-priority candidate from the graph abstraction, the execution layer can switch to the auxiliary IVF view to reduce resident metadata and make the access pattern closer to sequential list scans. Thus, the auxiliary IVF view complements graph search, providing a memory-efficient alternative when graph traversal would incur excessive out-of-core page misses under constrained memory.

5 System Optimization

The previous section described OrchANN's structure. We now show how this structure enables query-time optimizations by exploiting a workload property: cluster accesses are highly skewed, and their utility varies across clusters.

Key observation: workload skewness. Figure 6a shows that query accesses are highly concentrated across clusters.

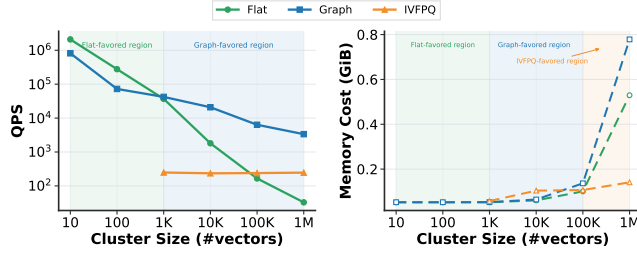
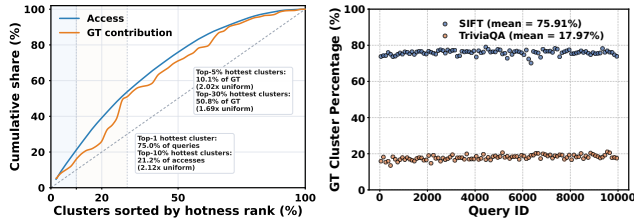


Figure 5. QPS and memory cost across index types vs. cluster size at Recall@100 $\geq$ 95%.



(a) Access and GT concentration. (b) GT clusters Percentage.

Figure 6. Runtime access hotness and cluster utility. A small set of clusters receives a large fraction of accesses and contributes disproportionately to ground-truth results, while many probed clusters, especially on semantic workloads, do not contribute to the final top- k results.

On HotpotQA, the hottest cluster appears in 75.0% of queries, and the top-10% hottest clusters receive 21.2% of all accesses, which is 2.12 $\times$ higher than a uniform distribution. More importantly, hotness is correlated with search utility rather than being only a cache-locality signal: the top-5% hottest clusters contribute 10.1% of ground-truth results, and the top-30% hottest clusters contribute 50.8% of ground-truth results. Thus, hot clusters are not merely frequently accessed; they are also more likely to contain useful neighbors.

Figure 6b further shows that routing alone is insufficient. Even after the routing stage produces candidate clusters, many probed clusters contain no ground-truth neighbors. On SIFT, about 24% of probed clusters do not contribute ground-truth results. On TriviaQA, the average ground-truth-cluster percentage drops to 17.97%, meaning that more than 80% of probed clusters do not contribute final top- k results. If these clusters are searched blindly, they still trigger out-of-core local-index accesses and SSD reads.

Together, these observations show that query hotness is both skewed and utility-aware. OrchANN therefore exploits cluster-level runtime signals in three ways. First, it incrementally adapts the graph abstraction with representative vectors from frequently useful clusters (Section 5.1). Second, it prioritizes promising clusters and prunes low-value accesses during online search (Section 5.2). Third, it schedules batched queries with similar cluster-access patterns close to each other to improve temporal locality in the page cache

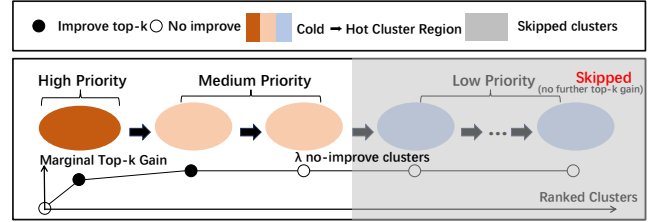


Figure 7. Graph abstraction guided cluster pruning.

and local index state (Section 5.3). These optimizations reduce redundant SSD accesses and improve locality without changing the partition layout.

5.1 Query-Aware Graph Abstraction Adaptation

OrchANN adapts $\mathcal{GA}$ using cluster-level query hotness. During serving, each worker records how often a cluster is selected by graph routing or searched by the local index in the current batch. At the end of batch t , OrchANN aggregates these counters and updates the hotness of cluster C_i as

$$h_i^{(t+1)} = \lambda h_i^{(t)} + (1 - \lambda) \cdot a_i^{(t)},$$

where $a_i^{(t)}$ is the batch-level access count and λ controls history decay. Every Δ_B batches, OrchANN incrementally re-allocates representative vectors based on cluster hotness: hot clusters receive new representatives, while representatives from cold clusters are retired. This update is lightweight because each insertion is performed entirely in memory and is comparable to an HNSW search over $\mathcal{GA}$; retired vectors are removed lazily by marking them inactive. To avoid blocking serving queries, the update is performed by a background thread on a shadow copy of $\mathcal{GA}$ and published through an atomic pointer swap; queries therefore traverse immutable snapshots. Since only a small fraction of vectors are adjusted in each round, the maintenance cost is amortized under concurrent serving. Figure 4 illustrates this process: compared with a static graph abstraction, the query-aware design re-allocates representatives from cold clusters to hot clusters. This replacement policy keeps $\mathcal{GA}$ bounded while shifting representative capacity toward frequently accessed clusters.

5.2 Cluster Pruning Guided by Graph Abstraction

During query processing, OrchANN traverses the in-memory graph abstraction and obtains a set of probe vectors (Figure 4, step 2). Since each probe vector is mapped to a physical cluster, these probes provide lightweight cluster-level distance proxies before any out-of-core local search is invoked. For a cluster C_i , let $\mathcal{P}_i(q)$ denote the set of probe vectors reached by query q and mapped to C_i . OrchANN defines the cluster distance by the closest probe in this set:

$$D_i(q) = \min_{x \in \mathcal{P}_i(q)} \text{dist}(q, x).$$

Clusters are then visited in ascending order of $D_i(q)$ (Figure 4, step 3), so a cluster is searched earlier when at least one of its graph-abstraction representatives is close to the query. This avoids ranking clusters by $|\mathcal{P}_i(q)|$, which can bias the search toward clusters with more probes but weaker geometric relevance. The pruning policy is independent of the local index choice, and therefore applies uniformly to Flat, graph, and auxiliary-IVF structures.

After reordering, OrchANN applies an early stop rule to avoid scanning the low-priority tail. Let M be the number of candidate clusters and let $n = f(M)$, for example $n = \lceil \rho M \rceil$ for a fixed ρ . During cluster processing, OrchANN maintains the global top- k queue. If the next n consecutively processed clusters do not improve the global top- k queue, OrchANN terminates cluster processing and skips the remaining candidates, as illustrated in Figure 7.

As shown in Section 6.2, detailed ablation experiments confirm the effectiveness of this pruning strategy: many clusters with large proxy distances are never loaded from SSD, reducing unnecessary out-of-core searches.

5.3 Query-Locality Scheduling

Query access skewness causes many queries to repeatedly visit a small set of hot clusters. If queries are processed strictly in arrival order, visits to the same cluster may be separated by unrelated queries, forcing the system to reload similar local-index metadata and vector pages from SSD. OrchANN therefore applies batch-level query-locality scheduling, which reorders nearby queries so that queries likely to access the same cluster are executed close together. This improves temporal locality in the page cache and local index state without changing any per-query search procedure.

Given a scheduling window of W queries, OrchANN first obtains a lightweight cluster visit plan for each query using the memory-resident graph abstraction $\mathcal{GA}$. This planning step runs in parallel and reuses the routing information already needed by normal query execution. For each query q , OrchANN extracts the first cluster in its ordered cluster list, denoted as $\text{top1}(q)$, and stably sorts queries within the window by $\text{top1}(q)$. Queries with the same key preserve their original arrival order. The window size W bounds scheduling delay and prevents global reordering, making the optimization suitable for high-throughput serving.

After scheduling, OrchANN executes each query normally with the same search parameters, pruning rule, and local index procedure. The scheduler only changes the order of independent queries. By placing queries with similar cluster visits closer together, OrchANN improves reuse of recently accessed local-index metadata and vector pages, thereby reducing redundant SSD traffic. This optimization is complementary to cluster pruning: pruning removes low-value cluster accesses, while query-locality scheduling improves locality among the clusters that are still searched.

Table 1. Characteristics of datasets.

Dataset	Vector	Dimension	Size	Data Type
HotpotQA	1.3M	384	2 GB	Float
SIFT	100M	128	12 GB	UInt8
TriviaQA	8.6M	768	25 GB	Float
DEEP	100M	96	36 GB	Float
SPACEV	1B	100	372 GB	Float

6 Evaluation

Implementation details. We implement OrchANN as a C++17 CPU-SSD out-of-core search engine built with CMake. The codebase contains roughly 12K lines of project code, excluding third-party and generated code. OrchANN uses FAISS [25] for k -means clustering and auxiliary IVF-view indexes, HNSWLib [31] for the graph abstraction, and NSG [12] for local graph indexes inside clusters. HNSWLib is a natural fit for the graph abstraction because it provides efficient in-memory graph search and supports incremental insertion; stale abstraction nodes are invalidated lazily during refresh. Cluster vectors, id mappings, and local index artifacts are stored as disk files and accessed through memory mapping at query time. Unless otherwise stated, we use $\alpha = 2.0$, $\beta = 1.0$, and $\rho = 0.7$ in Algorithm 1. For auxiliary-IVF construction, we tune the memory-budget fraction ρ_{aux} empirically; under the default setting this typically builds auxiliary IVF views for about 5%–10% of clusters. For cluster pruning, we use an early-stop window $n = \lceil \rho_{\text{prune}} M \rceil$ with $\rho_{\text{prune}} = 0.2$, where M is the number of candidate clusters.

Settings. All experiments are conducted on a server with two 64-core Intel Xeon Gold CPUs (2.5 GHz), 512 GB of DRAM, and a 1.92 TB SATA SSD, running Ubuntu 20.04 LTS. Unless otherwise stated, all SSD-resident datasets and index artifacts are placed on this SSD for all methods. Following the cgroup-based memory-constrained methodology used in Starling [48], we run each method inside a Linux cgroup v2 memory controller with `memory.high=4 GB` and `memory.max=10 GB` by default. This setting applies to both anonymous memory and resident file-backed pages, so memory mapped vector and index pages are charged to the same budget after being faulted in. We monitor cgroup memory usage. For major-page-fault measurements, we read the `pgmajfault` field from the cgroup’s `memory.stat` before and after each measurement run, and report per-query deltas. By default, query serving uses 48 worker threads.

Datasets. We evaluate OrchANN on five public datasets ranging from millions to billions of vectors, each with 10K queries: HotpotQA [54], SIFT [21], TriviaQA [26], DEEP [9], and SpaceV1B (SPACEV) [3] (Table 1). Across all datasets, IVF partitioning exhibits clear cluster-size skewness, while QA corpora such as HotpotQA and TriviaQA show particularly strong semantic skewness in addition to a pronounced long-tail distribution. We use a tighter 1 GB memory limit for

HotpotQA. For SPACEV, we convert the original `int8` vectors to `float` to align with common graph-based indexing libraries and ensure a fair comparison.

ANNS Baselines. We compare OrchANN against state-of-the-art out-of-core ANNS systems:

- **DiskANN** [20]: A foundational SSD-based ANNS system and one of the first to support billion-scale vector search.
- **Starling** [48]: An enhanced variant of DiskANN with improved indexing and navigation strategies.
- **SPANN** [4]: An IVF-based ANNS system that trades disk space for performance by storing multiple replicas of the dataset to improve accuracy and latency.

For all baselines, we use the best or default configurations recommended by their open-source implementations, and sweep only the standard search-strength parameter to obtain different recall points [20, 48]. For DiskANN and Starling, we fix the graph-search beam width to 4 and sweep the candidate-pool length L . All methods are run on the same machine as OrchANN. We also include **PipeANN** [17], a pipelined out-of-core ANNS engine that overlaps SSD I/O and computation.

6.1 Comparison With SOTA

As shown in Figures 8 and 9, we compare OrchANN against Starling [48], DiskANN [20], and SPANN [4] on five datasets: SIFT, DEEP, SPACEV, TriviaQA, and HotpotQA. OrchANN consistently achieves higher QPS and lower latency across recall targets. On feature datasets such as SIFT and DEEP, OrchANN delivers **2.0×–4.7×** higher QPS than Starling/DiskANN and reduces latency by up to **12.3×** compared with SPANN. The gains are larger on semantic workloads: on TriviaQA and HotpotQA, OrchANN achieves up to **17.2×** higher QPS and **25.0×** lower latency than SPANN at Recall@100 = 0.85.

These improvements come from reducing unnecessary SSD work at multiple levels. Compared with SPANN, OrchANN avoids replicated posting-list accesses by keeping raw vectors in non-replicated SSD partitions and providing boundary reachability through memory-resident logical overlap. Compared with DiskANN and Starling, OrchANN avoids relying only on SSD-resident graph traversal or static routing: its representative-vector graph abstraction produces stronger partition evidence, and the execution layer uses this evidence to prioritize partitions and prune low-utility accesses. Finally, scale-aware local indexes avoid using one fixed structure for clusters with different sizes and memory footprints.

The benefit is most pronounced on HotpotQA and TriviaQA, where dense semantic regions and skewed partitions amplify the I/O cost of coarse routing and replication. The advantage also extends to regular feature datasets: on SIFT, OrchANN achieves up to **2.7×** QPS gain, showing that the architecture improves robustness across both skewed semantic workloads and traditional feature benchmarks.

Table 2. I/O behavior of architecture skeletons. P, G, and O denote Partition, Graph, and our partition-local index substrate, respectively; O is not the full OrchANN pipeline.

Dataset	Mem.	Miss/q			Miss rate		
		P	G	O	P	G	O
SIFT	90%	1009.40	498.42	38.41	45.23%	18.47%	3.15%
SIFT	50%	1865.57	1392.12	131.99	83.59%	51.58%	10.84%
TriviaQA	90%	6204.83	60.10	21.85	2.74%	2.79%	1.58%
TriviaQA	50%	31502.00	145.39	36.68	13.89%	6.76%	2.66%

We also compare OrchANN with PipeANN [17] on SIFT and HotpotQA, as shown in Figure 10. On SIFT, OrchANN achieves up to **5.2×** higher QPS at Recall@100 = 0.9. On HotpotQA, it consistently delivers around **2×** higher QPS across the evaluated recall range.

6.2 Ablation Study

We first isolate the I/O behavior of the architectural substrate motivated in Section 3.1, and then decompose OrchANN into three parts that correspond to its hierarchical design: (1) the memory-resident graph abstraction and its cluster-level pruning support, (2) the out-of-core partition layer with hybrid local access paths, and (3) query-locality scheduling in the execution layer. Unless otherwise stated, component-level ablations are conducted on TriviaQA [26].

Architectural substrate: out-of-core I/O pattern. Table 2 compares three pure architecture skeletons under different memory limits. Partition uses IVF-style partition search, Graph uses a monolithic HNSW-style graph layout, and Ours uses disjoint SSD partitions with local graph indexes inside selected partitions. This experiment isolates the physical substrate rather than the full OrchANN pipeline.

The partition-local graph substrate consistently reduces unnecessary out-of-core accesses. On SIFT, Ours reduces misses/query from 1009.4 for Partition and 498.4 for Graph to 38.4 under the 90% memory limit; under the 50% memory limit, it reduces misses/query from 1865.6 and 1392.1 to 132.0. On TriviaQA, Ours also achieves the lowest miss rate under both memory limits, reducing misses/query to 21.9 and 36.7 under 90% and 50% memory, respectively. These results show that partition-local graph search preserves coarse partition locality while avoiding exhaustive partition scans and global random graph traversal. Therefore, disjoint SSD partitions with local indexes provide a suitable substrate for OrchANN.

Upper layer: graph abstraction and cluster pruning. Compared with centroid-only or random-sample routing, OrchANN’s representative-vector graph abstraction provides stronger cluster evidence by capturing off-center cluster shape. All variants are evaluated around the same 0.90 recall target. Enabling pruning is nearly lossless: the recall drop is below 0.001, and the pruned variants still remain above 0.90 recall. More generally, cluster pruning exposes an accuracy–efficiency tradeoff: even when a more aggressive

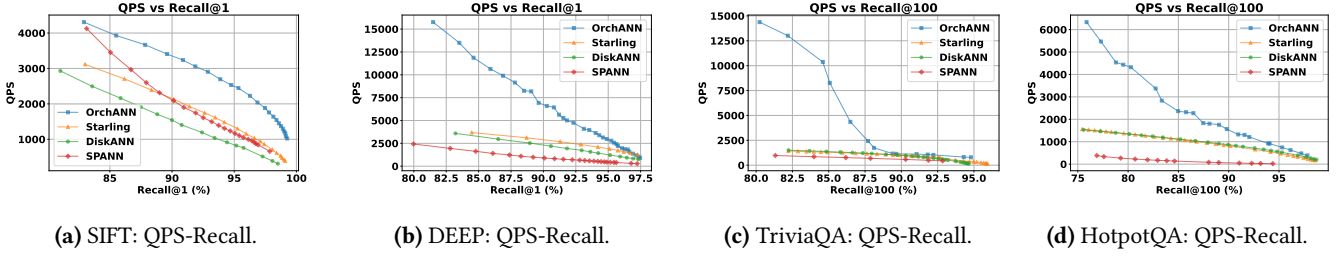


Figure 8. QPS overall evaluation on SIFT, DEEP, TriviaQA and HotpotQA datasets.

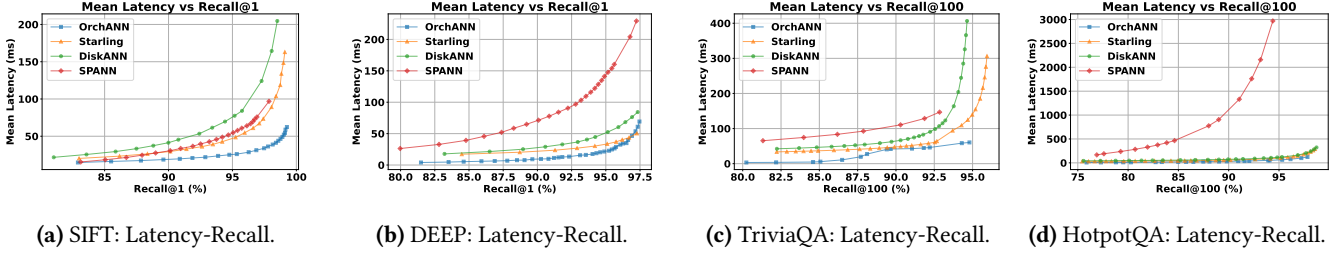


Figure 9. Latency overall evaluation on SIFT, DEEP, TriviaQA and HotpotQA datasets.

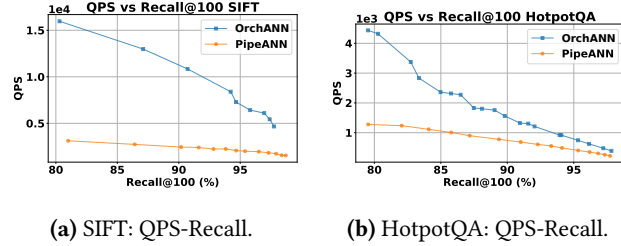


Figure 10. QPS evaluation comparing with PipeANN.

policy becomes mildly lossy, the saved SSD budget can be re-allocated to deeper local search or more candidate vectors in high-priority clusters instead of low-value partitions. At this recall level, OrchANN without pruning searches 16.0 clusters/query, 2.2 $\times$ fewer than random-sample routing; with pruning enabled, this drops to 8.1 clusters/query, 3.4 $\times$ fewer than random-sample routing with pruning. These results show that better cluster evidence improves QPS and latency, and also makes pruning more effective by ranking useful clusters earlier before SSD access.

Figure 11b separates the effect of dynamic updates. Dynamic GA alone improves over the static abstraction, and Dynamic+P performs best. This is because query-hot clusters are frequently visited and more likely to contribute useful results, so refreshing their representative vectors improves the ranking of candidate clusters before SSD access. We also measure cluster-selection precision and F1 score during query processing. Initial precision and F1 are about 0.16 and 0.24; after query-aware updates, they rise to about 0.20 and 0.30. This indicates that the graph abstraction becomes more

accurate over time, reducing unnecessary cluster probes and strengthening the pruning signal without sacrificing recall.

Out-of-core layer: hybrid local access paths. Figure 12 compares OrchANN’s hybrid indexing against uniform local-index choices under 90%, 75%, and 50% memory budgets. Hybrid indexing consistently achieves the highest QPS across recall targets, and the advantage becomes more important as memory becomes tighter. All-Graph performs well when its working set can be reused, but it suffers under memory pressure because graph metadata and neighbor traversal touch many random pages. All-Flat avoids metadata but becomes inefficient once clusters grow beyond the tiny-partition regime. All-IVF provides a more compact access pattern, but using it uniformly sacrifices the accuracy-oriented behavior of graph search on medium clusters. OrchANN avoids these extremes by using Flat for tiny clusters, Graph as the primary high-accuracy path, and memory-efficient auxiliary IVF views only for large clusters whose graph working set is likely to cause page-cache pressure.

Execution layer: query-locality scheduling. Figure 13 shows the effect of query-locality scheduling under multiple memory budgets. Enabling QS consistently improves QPS and reduces mean latency compared with processing queries in arrival order. The improvement is more visible under tighter memory budgets, where reusing recently touched local-index pages is more important. Since QS does not change per-query search parameters, the gain comes from better temporal locality among cluster accesses rather than from changing the search space. This optimization is complementary to cluster pruning: pruning reduces which clusters

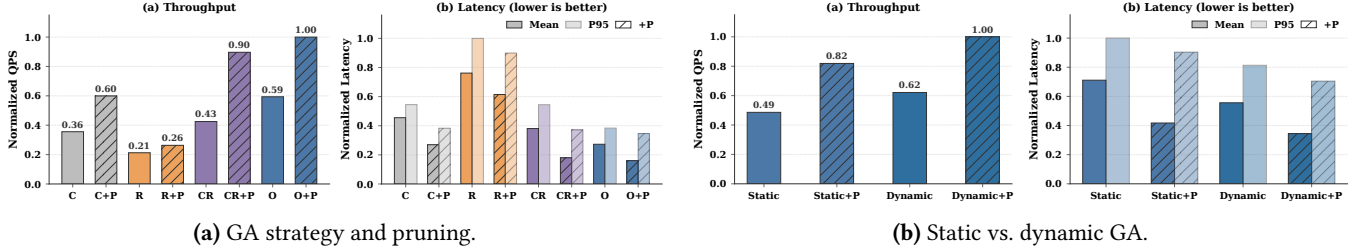


Figure 11. Ablation of graph abstraction and pruning. C, R, CR, and O denote centroid-only, random-sample, centroid+random, and OrchANN’s representative-vector graph abstraction, respectively; +P enables pruning.

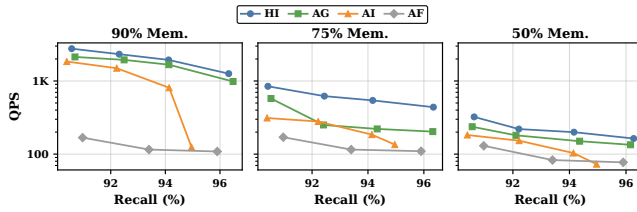


Figure 12. Ablation of the out-of-core partition layer under different memory budgets. HI denotes OrchANN’s hybrid indexing, while AG, AI, and AF denote all-graph, all-IVF, and all-Flat variants.

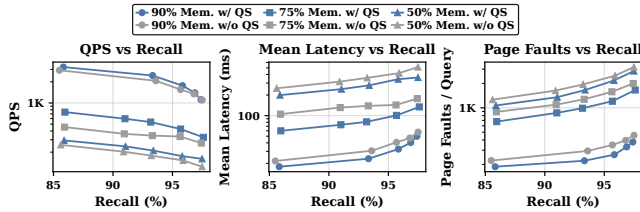


Figure 13. Ablation of query-locality scheduling under different memory budgets. QS reorders nearby queries with similar cluster visits to improve temporal locality without changing per-query search parameters.

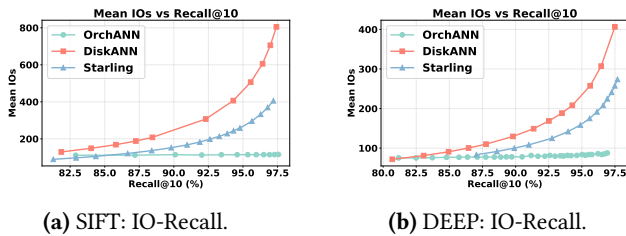


Figure 14. Page-access comparison on different datasets.

are accessed, while scheduling improves the order in which the remaining cluster accesses are issued.

6.3 I/O Analysis

We evaluate I/O efficiency by measuring the average number of page accesses per query for all systems. Figure 14a and Figure 14b compare OrchANN with DiskANN [20] and

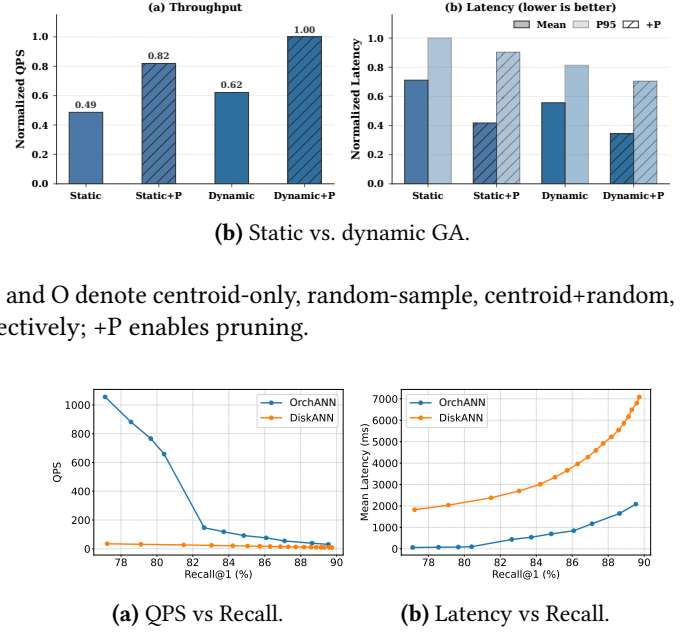


Figure 15. Performance evaluation on billion-scale data.

Starling [48]. We omit SPANN [4] from the plotted curves because its page-access counts are much larger and would obscure the other systems. On DEEP, SPANN already accesses 584 pages/query at Recall@10 = 0.928, and this number grows to 964 and 1,905 pages/query at Recall@10 = 0.958 and 0.982, respectively; these values exceed the plotted range needed to compare the remaining systems.

At Recall@10 = 97%, OrchANN achieves the same accuracy with fewer page accesses, requiring $3.5\times$ fewer accesses than Starling and $7\times$ fewer than DiskANN. The reduction comes from the hierarchical design: the memory-resident routing layer provides more accurate cluster hints before SSD access, while the out-of-core partition layer avoids a one-size-fits-all local index and lowers the cost of searching each cluster.

OrchANN also keeps page accesses stable as recall increases. From Recall@10 = 90% to 98%, its I/O grows by less than 10%. This stability indicates that higher recall does not require blindly probing many more disk-resident regions. Instead, logical cross-partition connectivity guides queries to relevant partitions, and cluster-level pruning skips low-utility candidates before issuing local searches. In contrast, DiskANN and Starling must touch increasingly more pages as recall targets rise, because their disk search paths or static routing signals provide weaker control over which out-of-core regions are actually useful.

6.4 Evaluation on Billion-Scale Data

Most existing ANNS systems, such as SPANN [4], require substantial memory during index construction, making them impractical for billion-scale datasets under limited hardware.

Metric	SPANN	Starling	DiskANN	OrchANN
Build time (s)	12,113	11,432	5,095	6,793
Storage (GB)	230.3	77.9	64.6	44.9

Table 3. Index construction time and disk storage overhead on DEEP.

For example, Starling [48] failed to finish index construction on the SPACEV dataset within 120 hours. In contrast, OrchANN is designed for resource-constrained environments and operates under a strict 10 GB memory cap.

Figure 15a and Figure 15b report QPS and latency under these constraints. At Recall@1 = 0.8, OrchANN achieves a **24.1×** higher QPS than DiskANN, and even at Recall@1 = 0.9 it sustains a **5×** advantage. Latency shows a similar trend: at comparable recall targets, OrchANN delivers up to **3.5×** lower mean latency. These results demonstrate that OrchANN remains effective at a billion scale and under tight DRAM limits, allowing OrchANN to maintain high efficiency where existing systems degrade sharply.

6.5 Index Cost

Index construction cost. Table 3 compares index construction time on the DEEP [9] dataset. For OrchANN, the build pipeline consists of partition construction, graph-abstraction generation, and per-cluster local index construction. The memory-resident graph abstraction is lightweight to build, while local indices are constructed independently for each partition, enabling scalable parallelism across clusters.

The additional cost of hybrid local access paths is small. Building the compact auxiliary IVF views for large clusters adds only about **272 s** to the total construction pipeline. Overall, OrchANN completes indexing in **6,793 s**, only slightly slower than DiskANN [20], while providing substantially higher query throughput and lower latency during serving.

Starling [48] and SPANN [4] incur higher build overhead due to more complex data layouts. SPANN is the slowest (**12,113 s**), largely because vectors are replicated across multiple partitions and the corresponding posting lists must be constructed with this redundancy. By keeping partitions physically independent while avoiding vector replication, OrchANN achieves a **1.78×** faster build than SPANN.

Disk storage overhead. Table 3 also compares disk usage on DEEP. OrchANN has the smallest footprint at **44.9 GB**, including vectors and indices. SPANN requires **230.3 GB**, more than **5×** larger, because vector replication substantially increases storage. DiskANN uses quantized representations and graph metadata to reduce memory pressure, but still needs additional index structures for high recall. Starling is more compact than SPANN, yet its block-aligned layout adds redundancy for locality.

In contrast, OrchANN stores each vector once and adds only a compact $\mathcal{GA}$. Even at large scale, the $\mathcal{GA}$ occupies only about **100 MB**, negligible compared with the vector

store. This duplication-free design keeps storage overhead low while preserving accuracy and out-of-core efficiency.

6.6 Parameter Sensitivity

We evaluate parameter sensitivity and scalability by varying both the recall target (Recall@1 vs. Recall@10) and the number of query threads on DEEP [9]. When moving from Recall@1 to Recall@10, all systems experience a QPS drop of roughly 20%. Despite this, OrchANN consistently achieves the highest QPS and the lowest latency across the entire recall range from 80% to 98%.

At Recall@10 = 90%, OrchANN delivers **3.5×** higher QPS than Starling [48], and at Recall@1 = 90%, it achieves **3.2×** higher QPS than Starling. In multi-threaded settings, OrchANN also scales effectively: with 64 threads, it outperforms Starling by **2.2×** and SPANN [4] by **5.4×** in QPS at Recall@10.

In terms of latency, OrchANN exhibits strong robustness to parameter changes, maintaining low latency and high throughput under varying recall targets and concurrency levels, which confirms its scalability and efficiency.

7 Related Work

While out-of-core ANNS systems are discussed separately in Section 2, prior work mainly includes: (1) algorithmic indexing optimized for low-latency search when the index fits in RAM, and (2) accelerator-based systems that leverage specialized hardware such as GPUs and PIM to improve throughput. For billion-scale settings where vectors exceed memory, existing out-of-core designs typically rely on IVF-based indices [4, 7, 53, 61], disk-resident graph indices [12, 32], or hierarchical structures with a small in-memory navigator over SSD-resident data [4, 20, 43].

Algorithmic ANNS solutions. A large body of work studies indexing strategies for in-memory ANNS [13–15, 32, 34, 41, 45, 47, 49, 60, 62]. HNSW [31] is widely adopted for its multi-layer graph structure, and methods such as SPTAG [3] combine space partitioning with graph traversal to reduce distance computations. These systems achieve excellent latency and recall, but they assume that both vectors and index metadata fit in RAM, which limits scalability.

Accelerator-based ANNS solutions. Recent systems exploit GPU-based [16, 36, 57, 62], PIM-based [5, 29, 50], FPGA-based [23], and CXL-based [19] designs. GPUs accelerate graph search and IVF+PQ filtering, but limited device memory constrains dataset size, and CPU–GPU communication can become a bottleneck even with overlapping techniques such as RUMMY [61]. Tree-based designs such as GTS [62] improve GPU utilization but still assume that the working set stays largely in memory. PIM-based approaches reduce data movement but remain restricted by hardware availability and limited support for complex index structures.

Together, these methods are effective with sufficient DRAM, GPU HBM, or specialized hardware, but do not directly address CPU–SSD search under tight memory budgets, where most vectors and index structures remain SSD-resident.

8 Conclusion

We presented OrchANN, a hierarchical out-of-core ANNS engine that decouples physical locality from logical connectivity. By keeping vectors in non-replicated SSD partitions, providing boundary reachability through a compact memory-resident graph abstraction, and combining scale-aware local indexes with utility-aware execution, OrchANN reduces redundant SSD work without sacrificing recall. Across five datasets under strict memory constraints, OrchANN achieves up to **17.2×** higher QPS and **25.0×** lower latency than state-of-the-art out-of-core baselines.

References

- [1] Adebawale Jeremy Adetayo, Mariam Oyinda Aborisade, and Basheer Abiodun Sanni. 2024. Microsoft Copilot and Anthropic Claude AI in education and library service. *Library Hi Tech News* (2024).
- [2] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. *Proc. ACM Manag. Data* 3, 3, Article 136 (June 2025), 28 pages. doi:10.1145/3725273
- [3] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. SPTAG: A library for fast approximate nearest neighbor search.
- [4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. Spann: Highly-efficient billion-scale approximate nearest neighborhood search. *Advances in Neural Information Processing Systems* 34 (2021), 5199–5212.
- [5] Sitian Chen, Amelie Chi Zhou, Yucheng Shi, Yusen Li, and Xin Yao. 2024. MemANNS: Enhancing Billion-Scale ANNS Efficiency with Practical PIM Hardware. *arXiv preprint arXiv:2410.23805* (2024).
- [6] Weijian Chen, Haotian Liu, Yangshen Deng, Long Xiang, Liang Huang, Gezi Li, and Bo Tang. 2026. AlayaLaser: Efficient Index Layout and Search Strategy for Large-scale High-dimensional Vector Similarity Search. *CoRR* abs/2602.23342 (2026). arXiv:2602.23342 doi:10.48550/ARXIV.2602.23342
- [7] Yaoqi Chen, Ruicheng Zheng, Qi Chen, Shuotao Xu, Qianxi Zhang, Xue Wu, Weihao Han, Hua Yuan, Mingqin Li, Yujing Wang, et al. 2024. Onesparse: A unified system for multi-index vector search. In *Companion Proceedings of the ACM Web Conference 2024*. 393–402.
- [8] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, et al. 2024. Magicpig: Lsh sampling for efficient llm generation. *arXiv preprint arXiv:2410.16179* (2024).
- [9] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [10] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonello, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 719–729.
- [11] Yangshen Deng, Zhengxin You, Long Xiang, Qilong Li, Peiqi Yuan, Zhaoyang Hong, Yitao Zheng, Wanting Li, Runzhong Li, Haotian Liu, et al. 2025. AlayaDB: The Data Foundation for Efficient and Effective Long-context LLM Inference. *arXiv preprint arXiv:2504.10326* (2025).
- [12] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754
- [13] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [14] Jianyang Gao and Cheng Long. 2024. RaBitQ: quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [15] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [16] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik PA Lensch. 2022. Ggnn: Graph-based gpu nearest neighbor search. *IEEE Transactions on Big Data* 9, 1 (2022), 267–279.
- [17] Hao Guo and Youyou Lu. 2025. Achieving {Low-Latency}{Graph-Based} Vector Search via Aligning {Best-First} Search Algorithm with {SSD}. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 171–186.
- [18] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. 2025. HedraRAG: Co-Optimizing Generation and Retrieval for Heterogeneous RAG Workflows. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 623–638.
- [19] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. {CXL-ANNS}:{Software-Hardware} collaborative memory disaggregation and computation for {Billion-Scale} approximate nearest neighbor search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 585–600.
- [20] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).
- [21] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57 The ANN_SIFT1M dataset was introduced in this paper. Dataset available at <http://corpus-texmex.irisa.fr/>.
- [22] Haodi Jiang, Hao Guo, Minhui Xie, Jiwei Shu, and Youyou Lu. 2025. High-Throughput, Cost-Effective Billion-Scale Vector Search with a Single GPU. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–27.
- [23] Wenqi Jiang, Shigang Li, Yu Zhu, Johannes De Fine Licht, Zhenhao He, Runbin Shi, Cedric Renggli, Shuai Zhang, Theodoros Rekatsinas, Torsten Hoefler, and Gustavo Alonso. 2023. Co-design Hardware and Algorithm for Vector Search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Denver, CO, USA) (SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 87, 15 pages. doi:10.1145/3581784.3607045
- [24] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. arXiv:2404.12457 [cs.DC] <https://arxiv.org/abs/2404.12457>
- [25] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [26] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. *arXiv preprint arXiv:1705.03551* (2017).

- arXiv:1705.03551 [cs.CL] <https://arxiv.org/abs/1705.03551v2>
- [27] Dingyi Kang, Dongming Jiang, Hanshen Yang, Hang Liu, and Bingzhe Li. 2025. Scalable Disk-Based Approximate Nearest Neighbor Search with Page-Aligned Graph. *CoRR* abs/2509.25487 (2025). arXiv:2509.25487 doi:10.48550/ARXIV.2509.25487
 - [28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, and Karpukhin. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
 - [29] Yiwei Li, Yuxin Jin, Boyu Tian, Huanchen Zhang, and Mingyu Gao. 2025. ANSMET: Approximate Nearest Neighbor Search with Near-Memory Processing and Hybrid Early Termination. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1093–1107.
 - [30] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. 2024. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516* (2024).
 - [31] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
 - [32] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. Parlayann: Scalable and deterministic parallel graph-based approximate nearest neighbor search algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 270–285.
 - [33] Jason Mohoney, Divesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025*, Li-dong Zhou and Yuanyuan Zhou (Eds.). USENIX Association, 153–169. <https://www.usenix.org/conference/osdi25/presentation/mohoney>
 - [34] Seth Ockerman, Amal Gueroudji, Song Young Oh, Robert Underwood, Nicholas Chia, Kyle Chard, Robert Ross, and Shivaram Venkataraman. 2025. Exploring Distributed Vector Databases Performance on HPC Platforms: A Study with Qdrant. In *Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 575–581.
 - [35] Shumpei Okura, Yukihiro Tagami, Shingo Ono, and Akira Tajima. 2017. Embedding-based news recommendation for millions of users. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 1933–1942.
 - [36] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4236–4247.
 - [37] OpenAI. 2025. OpenAI developer platform. <https://platform.openai.com/docs/overview>
 - [38] Aditya Pal, Chantat Eksombatchai, Yitong Zhou, Bo Zhao, Charles Rosenberg, and Jure Leskovec. 2020. PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 2311–2320. doi:10.1145/3394486.3403280
 - [39] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector database management techniques and systems. In *Companion of the 2024 International Conference on Management of Data*. 597–604.
 - [40] Konstantinos I Roulmliotis and Nikolaos D Tselikas. 2023. Chatgpt and open-ai models: A preliminary review. *Future Internet* 15, 6 (2023), 192.
 - [41] Tobias Rubel, Richard Wen, Laxman Dhulipala, Lars Gottesbüren, Rajesh Jayaram, and Jakub Lacki. 2026. PiPNN: Ultra-Scalable Graph-Based Nearest Neighbor Indexing. *CoRR* abs/2602.21247 (2026). arXiv:2602.21247 doi:10.48550/ARXIV.2602.21247
 - [42] Korakit Seemakhupt, Sihang Liu, and Samira Khan. 2024. EdgeRAG: Online-Indexed RAG for Edge Devices. *ArXiv* abs/2412.21023 (2024). <https://api.semanticscholar.org/CorpusID:275134434>
 - [43] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using {SmartSSDs}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 1135–1150.
 - [44] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuechang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards High-throughput and Low-latency Billion-scale Vector Search via {CPU/GPU} Collaborative Filtering and Re-ranking. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 171–185.
 - [45] Sairaj Voruganti and M Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.
 - [46] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
 - [47] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–29.
 - [48] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An i/o-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.
 - [49] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. Detlsh: a locality-sensitive hashing scheme with dynamic encoding tree for approximate nearest neighbor search. *arXiv preprint arXiv:2406.10938* (2024).
 - [50] Puqing Wu, Minhui Xie, Enrui Zhao, Dafang Zhang, Jing Wang, Xiao Liang, Kai Ren, and Yunpeng Chai. 2025. Turbocharge {ANNS} on Real {Processing-in-Memory} by Enabling {Fine-Grained} {Per-PIM-Core} Scheduling. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 1223–1241.
 - [51] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] <https://arxiv.org/abs/2308.08155>
 - [52] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 641–649.
 - [53] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. 2023. Spfresh: Incremental in-place update for billion-scale vector search. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 545–561.
 - [54] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

- [55] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [57] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated proximity graph approximate nearest Neighbor search and construction. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 552–564.
- [58] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. 2025. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–30.
- [59] Jianjin Zhang, Zheng Liu, Weihao Han, Shitao Xiao, Ruicheng Zheng, Yingxia Shao, Hao Sun, Hanqing Zhu, Premkumar Srinivasan, Weiwei Deng, et al. 2022. Uni-retriever: Towards learning the unified embedding based retriever in bing sponsored search. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4493–4501.
- [60] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.
- [61] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Vector Query Processing for Large Datasets Beyond {GPU} Memory with Reordered Pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 23–40.
- [62] Yifan Zhu, Ruiyao Ma, Baihua Zheng, Xiangyu Ke, Lu Chen, and Yunjun Gao. 2024. GTS: GPU-based tree index for fast similarity search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.